

Тема 5.2. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции. Алгоритми

Съдържание:

1. Quicksort

- Избиране на първия елемент
- Избиране на последния елемент
- Избиране на случаен елемент
- Избиране на медианата

2. Сортиране чрез сливане (Merge Sort)

3. Пирамидално сортиране (Heapsort)

Тема 5.2. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции. Алгоритми

1. **Quicksort (бързо сортиране)** е един от най-ефективните алгоритми за сортиране, който е базиран на принципа "разделяй и владей". Той функционира чрез избиране на "опорна точка" (pivot) и разделяне на елементите на масива на две части: елементите по-малки от опорната точка и елементите по-големи или равни на опорната точка. След това алгоритъмът рекурсивно сортира двете получени подмножества.

Постъпково изпълнение:

☐ Избор на опорна точка

От масива се избира елемент, служещ за опорна точка. Възможните стратегии за избора на "опорна точка" са:

- Избиране на първия елемент;
- Избиране на последния елемент;
- Избиране на случаен елемент;
- Избиране на медиана.

☐ Разделяне

След като е избрана опорната точка, масивът се разделя. Процесът на разделяне включва:

- Инициализиране на два указателя - един за по-малките елементи и един за по-големите.
- Преминаване през масива и преместване на елементите така, че частта от масива с по-малките елементи да бъде преди опорната точка, а частта с по-големите - след нея.

☐ Рекурсивно сортиране

След разделянето, Quicksort рекурсивно сортира двете подчасти на масива, всяка от които е сама по себе си с по-малките и с по-големите елементи.

☐ Сливане на резултатите

При достигане на рекурсивна база, когато подмасивите са малки (обикновено с един или нула елемента), рекурсивната функция се прекратява и масивът е вече сортиран.

Тема 5.2. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции. Алгоритми

1.1. Избиране на първия елемент за “опорна точка”

Пример:

```
defmodule QuickSort do
  def sort_first([], do: [])
  def sort_first([pivot | rest]) do
    less_than_pivot = Enum.filter(rest, fn x -> x <= pivot end)
    greater_than_pivot = Enum.filter(rest, fn x -> x > pivot end)
    # Обединение на сортираните части
    sort_first(less_than_pivot) ++ [pivot] ++ sort_first(greater_than_pivot)
  end
end
```

```
arr = [3, 6, 8, 10, 1, 2, 1]
sorted_arr = QuickSort.sort_first(arr)
IO.inspect(sorted_arr)
```

```
[1, 1, 2, 3, 6, 8, 10]
```

```
defmodule QuickSort do
  def sort_first([], do: [])
  def sort_first([pivot | rest]) do
    less_than_pivot = Enum.filter(rest, fn x -> x <= pivot end)
    greater_than_pivot = Enum.filter(rest, fn x -> x > pivot end)
    sort_first(less_than_pivot) ++ [pivot] ++ sort_first(greater_than_pivot)
  end
end
```

```
# Пример за използване
arr = [3, 6, 8, 10, 1, 2, 1]
sorted_arr = QuickSort.sort_first(arr)
IO.inspect(sorted_arr)
```

Тема 5.2. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции. Алгоритми

1.1. Избиране на първия елемент за “опорна точка”

1) `defmodule QuickSort do` - Тук се дефинира нов модул с име `QuickSort`. В Elixir, модулите се използват за организиране на кода и групиране на свързани функции.

2) `def sort_first([], do: [])` - Тук се дефинира функция на име `sort_first`, която приема един аргумент. Първият `def` указва, че това е началото на дефиницията на функцията. В този случай, функцията проверява дали списъкът е празен (`[]`). Ако е така, тя връща празен списък. Това е базовият случай за рекурсия, който предотвратява безкрайна рекурсия и указва какво да се прави, когато не останат елементи за сортиране.

3) `def sort_first([pivot | rest]) do` - Тук се дефинира друга версия на функцията `sort_first`, която приема списък, който не е празен. Тук, списъкът се разделя на два компонента: `pivot` (първият елемент на списъка) и `rest` (всички останали елементи в списъка). Тази структура е полезна за алгоритъма, защото `pivot` е стойността около, която ще се организират останалите елементи.

4) `less_than_pivot = Enum.filter(rest, fn x -> x <= pivot end)` - Тук, функцията `Enum.filter/2` се използва за създаване на нов списък, наречен `less_than_pivot`, който съдържа всички елементи от `rest`, които са по-малки или равни на `pivot`. `fn x -> x <= pivot end` е анонимна функция, която проверява всяко `x` от `rest`.

1.1. Избиране на първия елемент за “опорна точка”

5) `greater_than_pivot = Enum.filter(rest, fn x -> x > pivot end)` - Тук се създава списък `greater_than_pivot`, който съдържа всички елементи от `rest`, които са по-големи от `pivot`.

6) `sort_first(less_than_pivot) ++ [pivot] ++ sort_first(greater_than_pivot)` - В тази част от алгоритъма рекурсивно се извиква функцията `sort_first` за списъка `less_than_pivot`, за да се сортира. След това се добавя `pivot` в средата, и след това отново рекурсивно се извиква `sort_first` за списъка `greater_than_pivot`, за да се сортира и той. Операцията `++` се използва за конкатенация на списъците. По този начин, накрая получаваме обединен и сортиран списък от по-малките стойности, стойността на `pivot` и по-големите стойности.

7) `end` - това затваря блока на модула `QuickSort`.

8) `arr = [3, 6, 8, 10, 1, 2, 1]` - Тук се декларира списък с име `arr`, който съдържа несортирани числа.

9) `sorted_arr = QuickSort.sort_first(arr)` - Тук функцията `sort_first` от модула `QuickSort` се извиква с аргумент `arr`, и резултатът (сортирания списък) се запазва в променливата `sorted_arr`.

10) `IO.inspect(sorted_arr)` - `IO.inspect/1` извежда, или "инспектира", съдържанието на `sorted_arr` на конзолата, така че да можем да видим резултата от сортирането.

Обобщение

Като цяло, кодът реализира основния алгоритъм за бързо сортиране, извършвайки няколко стъпки: разделя списъка на елементи по-малки и по-големи от избраната "опорна точка", рекурсивно сортира тези подсписъци и след това ги обединява в един сортиран списък.

Тема 5.2. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции. Алгоритми

1.2. Избиране на последния елемент за “опорна точка”

Пример:

```
defmodule QuickSort do
  def sort_last([], do: [])
  def sort_last(arr) do
    pivot = List.last(arr)
    rest = List.delete_at(arr, -1) # премахваме последния елемент
    less_than_pivot = Enum.filter(rest, fn x -> x <= pivot end)
    greater_than_pivot = Enum.filter(rest, fn x -> x > pivot end)
    # Обединение на сортираните части
    sort_last(less_than_pivot) ++ [pivot] ++ sort_last(greater_than_pivot)
  end
end
```

```
arr = [3, 6, 8, 10, 1, 2, 1]
sorted_arr = QuickSort.sort_last(arr)
IO.inspect(sorted_arr)
```

```
[1, 1, 2, 3, 6, 8, 10]
```

```
defmodule QuickSort do
  def sort_last([], do: [])
  def sort_last(arr) do
    pivot = List.last(arr)
    rest = List.delete_at(arr, -1) # премахваме последния елемент
    less_than_pivot = Enum.filter(rest, fn x -> x <= pivot end)
    greater_than_pivot = Enum.filter(rest, fn x -> x > pivot end)
    sort_last(less_than_pivot) ++ [pivot] ++ sort_last(greater_than_pivot)
  end
end
```

```
# Пример за използване
arr = [3, 6, 8, 10, 1, 2, 1]
sorted_arr = QuickSort.sort_last(arr)
IO.inspect(sorted_arr)
```

Тема 5.2. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции. Алгоритми

1.2. Избиране на последния елемент за “опорна точка”

- 1) `defmodule QuickSort do` – Дефинира се нов модул с име `QuickSort`.
- 2) `def sort_last([], do: [])` - Тук се дефинира функцията `sort_last`. Първата версия на функцията проверява дали входният списък е празен (`[]`). Ако е така, функцията просто връща празен списък. Това е базовият случай за рекурсия и предотвратява безкрайна рекурсия.
- 3) `def sort_last(arr)` do – Тук се дефинира втора версия на функцията `sort_last`, която приема списък `arr`, който не е празен. В тази функция ще се реализира основната логика за бързо сортиране.
- 4) `pivot = List.last(arr)` - Тук се взима последния елемент от списъка `arr` и се определя като `pivot`. Функцията `List.last/1` извлича последния елемент, който след това служи като опорна стойност за сортиране.
- 5) `rest = List.delete_at(arr, -1)` *# премахваме последния елемент*
Получава се подсписъка, съдържащ всичките елементи освен `pivot`. Функцията `List.delete_at/2` премахва елемента от индекс `-1`, който е последният елемент на `arr`. Резултатът се запазва в `rest`.

Тема 5.2. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции. Алгоритми

1.2. Избиране на последния елемент за “опорна точка”

6) `less_than_pivot = Enum.filter(rest, fn x -> x <= pivot end)` - Тук се използва `Enum.filter/2`, за да се създаде нов списък `less_than_pivot`, който съдържа всички елементи от `rest`, които са по-малки или равни на `pivot`. Анонимната функция `fn x -> x <= pivot end` проверява всеки елемент `x` от `rest`.

7) `greater_than_pivot = Enum.filter(rest, fn x -> x > pivot end)` – По аналогичен начин се създава списък с име `greater_than_pivot`, който съдържа всички елементи от `rest`, които са по-големи от `pivot`.

8) *# Обединение на сортираните части* - Този коментар описва, че следващата линия ще обедини сортираните части.

9) `sort_last(less_than_pivot) ++ [pivot] ++ sort_last(greater_than_pivot)` - Тук се извиква `sort_last` рекурсивно за `less_than_pivot`, за да се сортира, след което се добавя `pivot` в средата и накрая отново се извиква `sort_last(greater_than_pivot)`, за новата сортировка. Операцията `++` конкатенира (обединява) списъците. Крайният резултат е обединен сортиран списък от по-малките стойности, стойността на `pivot` и по-големите стойности от `pivot-a`.

10) `end` - Затваря блока на функцията `sort_last`.

Тема 5.2. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции. Алгоритми

1.2. Избиране на последния елемент за **“опорна точка”**

11) `end` – Тук се затваря блока на модула `QuickSort`.

12) `arr = [3, 6, 8, 10, 1, 2, 1]` – Деклариране на списък `arr`, който съдържа несортирани числа.

13) `sorted_arr = QuickSort.sort_last(arr)` – Този ред извиква функцията `sort_last` от модула `QuickSort`, предавайки списъка `arr` за сортиране, и съхранява резултата (сортирания списък) в променливата `sorted_arr`.

14) `IO.inspect(sorted_arr)` - Тази функция (`IO.inspect/1`) извежда (или "инспектира") стойността на `sorted_arr` в конзолата, позволявайки да се види крайния резултат от сортирането.

Обобщение

Тук отново е реализиран алгоритъма на бързото сортиране. Той разделя списъка на две части (по-малките и по-големите от `pivot`), рекурсивно сортира тези подсписъци и след това ги обединява заедно с `pivot`, за да получи окончателно сортирания списък.

Тема 5.2. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции. Алгоритми

1.3. Избиране на случаен елемент за “опорна точка”. Тук има особеност при избора на случаен елемент при алгоритъма за бързо сортиране (Quick Sort), а именно, че случайният елемент може да не бъде уникален (или да бъде включен повече от веднъж в резултата), което може да доведе до неправилна логика при сортирането. Вместо да създаваме списъци с Enum.filter, тук ще използваме подход с разделяне, подобен на стандартната реализация на Quick Sort. Вместо Enum.filter се използва Enum.reduce за да разделим списъка на две части на базата на опорната точка. Списъците less_than_pivot и greater_than_pivot се попълват с помощта на акумулатор. Тази стъпка гарантира, че няма дублиране на елемента pivot. След това, при обединяване на сортираните части, се уверяваме, че опорният елемент pivot не се включва повторно в резултата (премахва се от greater_than_pivot).

Пример:

```
defmodule QuickSort do
  def sort_random([], do: [])

  def sort_random(arr) do
    pivot = Enum.random(arr)
    {less_than_pivot, greater_than_pivot} =
      Enum.reduce(arr, {[], []}, fn x, {less, greater} ->
        if x < pivot do
          {[x | less], greater}
        else
          {less, [x | greater]}
        end
      end)

    # Обединение на сортираните части
    sort_random(less_than_pivot) ++ [pivot]
    ++ sort_random(greater_than_pivot -- [pivot])
  end
end

arr = [3, 6, 8, 10, 1, 2, 1]
sorted_arr = QuickSort.sort_random(arr)
IO.inspect(sorted_arr)
```

```
defmodule QuickSort do
  def sort_random([], do: [])

  def sort_random(arr) do
    pivot = Enum.random(arr) # Избиране на случаен елемент като опорна точка
    {less_than_pivot, greater_than_pivot} =
      Enum.reduce(arr, {[], []}, fn x, {less, greater} ->
        if x < pivot do
          {[x | less], greater}
        else
          {less, [x | greater]}
        end
      end)

    # Обединяване на сортираните части
    sort_random(less_than_pivot) ++ [pivot] ++ sort_random(greater_than_pivot -- [pivot])
  end
end

# Пример за използване
arr = [3, 6, 8, 10, 1, 2, 1]
sorted_arr = QuickSort.sort_random(arr)
IO.inspect(sorted_arr)
```

[1, 1, 2, 3, 6, 8, 10]

Тема 5.2. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции. Алгоритми

1.3. Избиране на случаен елемент

- 1) `defmodule QuickSort do` – Този ред дефинира нов модул с име `QuickSort`.
- 2) `def sort_random([], do: [])` - Тук се дефинира функцията `sort_random`. Първата версия на функцията проверява дали входният списък е празен (`[]`). Ако е така, функцията просто връща празен списък. Това е базовият случай за рекурсия и предотвратява безкрайна рекурсия.
- 3) `def sort_random(arr) do` – Този ред дефинира втора версия на функцията `sort_random`, която приема списък `arr`, който не е празен. В тази функция се реализира основната логика за бързо сортиране.
- 4) `pivot = Enum.random(arr)` - Тук се използва функцията `Enum.random/1`, за да се избере произволен елемент от списъка `arr` и се задава като `pivot`. Изборът на произволен пивот може да помогне да се избегнат лоши случаи с производителността на алгоритъма.
- 5) `{less_than_pivot, greater_than_pivot} = Enum.reduce(arr, {[], []}, fn x, {less, greater} ->` - Тук се използва `Enum.reduce/3`, за да се премине през всеки елемент `x` от списъка `arr`. Началната стойност на редукцията е кортеж `{[], []}`, който съдържа два празни списъка - един за елементите, които са по-малки от `pivot`, и друг за елементите, които са по-големи или равни на `pivot`.

Тема 5.2. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции. Алгоритми

1.3. Избиране на случаен елемент

```
6) if x < pivot do
    {[x | less], greater}
else
    {less, [x | greater]}
end
```

В тялото на анонимната функция се извършва проверка дали текущият елемент x е по-малък от pivot . Ако е, се добавя x в списъка `less` (с по-малките стойности) с операцията `[x | less]`. Ако не е, се добавя x в списъка `greater` (с по-големите или равни стойности) с операцията `[x | greater]`. Всяка итерация на `reduce` актуализира `tuple` с новите списъци `less` и `greater`.

7) `end)` – С този ред се затваря тялото на анонимната функция, което служи за обработка на всеки елемент от `arg`.

8) *# Обединение на сортираните части* - Този коментар указва, че следващият ред ще обедини сортираните части.

9) `sort_random(less_than_pivot) ++ [pivot] ++ sort_random(greater_than_pivot -- [pivot])` - Този ред е основната част от алгоритъма. Тук:

Рекурсивно се извиква `sort_random` за `less_than_pivot`, за да се сортира. След това се добавя `pivot` в средата. В края, отново рекурсивно се извиква `sort_random` за `greater_than_pivot`, но преди това се премахва `pivot` от `greater_than_pivot`, за да се избегне дублирането му. Операцията `--` премахва елемента от списъка. Операцията `++` служи за конкатенация на тези списъци, за да получим окончателно сортиран списък.

10) `end` - Този ред затваря блока на функцията `sort_random`.

11) `end` - Този ред затваря блока на модула `QuickSort`.

Тема 5.2. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции. Алгоритми

1.3. Избиране на случаен елемент

12) `arr = [3, 6, 8, 10, 1, 2, 1]` - Тук се декларира списък `arr`, който съдържа несортирани числа.

13) `sorted_arr = QuickSort.sort_random(arr)` - В този ред се извиква функцията `sort_random` от модула `QuickSort`, като се предава списъка `arr` за сортиране, и се съхранява резултата (сортирания списък) в променливата `sorted_arr`.

14) `IO.inspect(sorted_arr)` – тази функция (`IO.inspect/1`) извежда (или "инспектира") стойността на `sorted_arr` в конзолата, така че да можем да видим крайния резултат от сортирането.

Обобщение

Този код реализира алгоритъма за бързо сортиране с произволно избран елемент за пивот. Той разделя списъка на две части (по-малките и по-големите от пивота), рекурсивно сортира тези подсписъци и след това ги обединява заедно с пивота, за да получи окончателно сортирания списък.

Тема 5.2. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции. Алгоритми

❑ Сложност

- **Средна и най-добра ситуация:** $O(n \log n)$, когато опорната точка разделя масива приблизително наполовина.
- **Най-лоша ситуация:** $O(n^2)$, когато опорната точка е най-малкия или най-големия елемент и масивът не е балансиран.

❑ Предимства и недостатъци

Предимства:

- Подходящ за големи данни.
- Често е по-бърз от другите алгоритми за сортиране, благодарение на малкия разход на памет и правилното управление на кеша в съвременните компютри.

Недостатъци:

- Може да е нестабилен (може да не запази реда на равни елементи).
- Най-лошата сложност е $O(n^2)$, но това може да бъде избегнато с правилния избор на опорна точка.
- Това е основната концепция за Quicksort. Алгоритъмът е широко използван поради своята ефективност и относително лесна реализация.

1.4. Избиране на медианата за “опорна точка”. Определяне на медианата: Използваме функцията `median_of_three`, за да минимизираме възможността за лошо представяне, като избираме медианата от първия, средния и последния елемент. **Разделяне на елементите:** Използваме `Enum.reduce`, за да генерираме три категории: елементи по-малки от медианата, равни на медианата и по-големи от медианата. **Рекурсивно сортиране:** Извикваме `sort` за списъците с елементи, по-малки и по-големи от медианата, и комбинираме резултата за окончателното сортиране.

Пример:

```
defmodule QuickSort do
  def sort([]), do: []

  def sort(arr) when length(arr) <= 1, do: arr

  def sort(arr) do
    pivot = median_of_three(arr)
    {less, equal, greater} = Enum.reduce(arr, {[], [], []}, fn x, {less, equal, greater} ->
      cond do
        x < pivot -> {[x | less], equal, greater}
        x > pivot -> {less, equal, [x | greater]}
        true -> {less, [x | equal], greater}
      end
    end)

    sort(less) ++ equal ++ sort(greater)
  end

  defp median_of_three(arr) do
    [a, b, c] = [List.first(arr), Enum.at(arr, div(length(arr), 2)), List.last(arr)]
    Enum.sort([a, b, c]) |> Enum.at(1)
  end
end

# Пример за използване
arr = [3, 6, 8, 10, 1, 2, 1]
sorted_arr = QuickSort.sort(arr)
IO.inspect(sorted_arr)
```

```
defmodule QuickSort do
  def sort([]), do: []

  def sort(arr) when length(arr) <= 1, do: arr

  def sort(arr) do
    pivot = median_of_three(arr)
    {less, equal, greater} = Enum.reduce(arr, {[], [], []}, fn x, {less, equal, greater} ->
      cond do
        x < pivot -> {[x | less], equal, greater}
        x > pivot -> {less, equal, [x | greater]}
        true -> {less, [x | equal], greater}
      end
    end)

    sort(less) ++ equal ++ sort(greater)
  end

  defp median_of_three(arr) do
    [a, b, c] = [List.first(arr), Enum.at(arr, div(length(arr), 2)), List.last(arr)]
    Enum.sort([a, b, c]) |> Enum.at(1)
  end
end

# Пример за използване
arr = [3, 6, 8, 10, 1, 2, 1]
sorted_arr = QuickSort.sort(arr)
IO.inspect(sorted_arr)
```

[1, 1, 2, 3, 6, 8, 10]

1.4. Избиране на медианата за “опорна точка”.

- 1) `defmodule QuickSort do` – Дефиниране на нов модул с име `QuickSort`.
- 2) `def sort([]), do: []` – Дефиниране на функцията `sort`, която проверява дали входният списък е празен (`[]`). Ако е така, функцията просто връща празен списък. Това е базов случай за рекурсия и предотвратява безкрайна рекурсия.
- 3) `def sort(arr) when length(arr) <= 1, do: arr` - Тази версия на функцията `sort` проверява дали дължината на входния списък е по-малка или равна на 1. Ако това е вярно, елементите не могат да бъдат сортирани (тъй като имаме 0 или 1 елемент), така че просто връща входния списък `arr` без промяна.
- 4) `def sort(arr) do` – Този ред дефинира основната версия на функцията `sort`, която ще обработва списъци с повече от един елемент. Тук ще се реализира основната логика на бързото сортиране.
- 5) `pivot = median_of_three(arr)` - Тук се извиква помощна функция `median_of_three`, която избира пивота, основавайки се на медианата на трите стойности: първият, средният и последният елемент на списъка `arr`. Пивотът е важен за ефективността на алгоритъма.
- 6) `{less, equal, greater} = Enum.reduce(arr, {[], [], []}, fn x, {less, equal, greater} ->` - Тук се използва `Enum.reduce/3`, за да се премине през всеки елемент `x` от списъка `arr`. Началната стойност на редукцията е кортеж `{[], [], []}`, който съдържа три празни списъка: `less` (с по-малките стойности), `equal` (с равните на пивота) и `greater` (с по-големите стойности).

1.4. Избиране на медианата за “опорна точка”.

7) cond do

$x < \text{pivot} \rightarrow \{[x \mid \text{less}], \text{equal}, \text{greater}\}$

$x > \text{pivot} \rightarrow \{\text{less}, \text{equal}, [x \mid \text{greater}]\}$

 true $\rightarrow \{\text{less}, [x \mid \text{equal}], \text{greater}\}$

end

В тялото на анонимната функция, cond за проверка на следните условия:

- Ако x е по-малък от пивота (pivot), добавяме x в списъка less.
- Ако x е по-голям от пивота, добавяме x в списъка greater.
- В противен случай, x е равен на пивота и го добавяме в списъка equal.

Всяка итерация на reduce актуализира кортежа с новите списъци less, equal и greater.

8) end) – Реда затваря тялото на анонимната функция, която обработва всеки елемент от arr.

9) sort(less) ++ equal ++ sort(greater) - Тук рекурсивно се извиква sort за списъка less, след което се добавя списъка equal (елементи равни на пивота), и накрая рекурсивно се извиква sort за списъка greater. Операцията ++ служи за конкатенация на трите списъка, за да се получи окончателно сортирания списък.

10) end – С този ред се затваря блока на функцията sort.

11) defp median_of_three(arr) do – Тук се дефинира помощна функция median_of_three, която е частна и се използва само в рамките на модула QuickSort.

1.4. Избиране на медианата за “опорна точка”.

12) `[a, b, c] = [List.first(arr), Enum.at(arr, div(length(arr), 2)), List.last(arr)]` - Тук се извличат три стойности от списъка `arr`:

- Първият елемент (`List.first(arr)`).
- Средният елемент (`Enum.at(arr, div(length(arr), 2))`), който се намира на индекс, равен на половината от дължината на списъка.
- Последният елемент (`List.last(arr)`).

Тези три стойности се записват в списъка `[a, b, c]`.

13) `Enum.sort([a, b, c]) |> Enum.at(1)` - След това, се сортират трите стойности (`[a, b, c]`) с `Enum.sort/1` и се взима вторият елемент (`Enum.at(1)`), който е медианата. Тази стойност ще бъде върната като пивот.

14) `end` – Този ред затваря блока на функцията `median_of_three`.

15) `end` – Този ред затваря блока на модула `QuickSort`.

16) *# Пример за използване* `arr = [3, 6, 8, 10, 1, 2, 1]` – Тук се декларира списък `arr`, който съдържа несортирани числа.

17) `sorted_arr = QuickSort.sort(arr)` - В този ред се извиква функцията `sort` от модула `QuickSort`, предавайки списъка `arr` за сортиране, и се съхранява резултата (сортирания списък) в променливата `sorted_arr`.

18) `IO.inspect(sorted_arr)` - И накрая, в този ред функцията `IO.inspect/1` се извежда (или "инспектира") стойността на `sorted_arr` в конзолата, позволявайки визуализация на крайния резултат от сортирането.

Обобщение

Този код реализира алгоритъма за бързо сортиране, като използва медианата на трите стойности (първия, средния и последния елемент на списъка) за определяне на пивота. Алгоритъмът разделя списъка на три части (по-малките от пивота, равните на пивота и по-големите от пивота), рекурсивно сортира тези части и след това ги обединява, за да получи окончателно сортирания списък. Дефиницията на медианата помага за по-добро представяне при различни входни данни.

2. Сортиране чрез сливане (Merge Sort). Алгоритмът разделя масива и след това слива подмасивите. Сортирането чрез сливане е рекурсивен алгоритъм, който разделя входния масив на две половини, сортира всяка половина и след това ги слива обратно заедно. Сортирането чрез сливане е един от най-ефективните алгоритми за сортиране с

Времева сложност:

Във всички случаи: $O(n \log n)$

Пример:

```
defmodule MergeSort do
  def merge_sort([], do: [])
  def merge_sort([single]), do: [single]
  def merge_sort(arr) do
    mid = div(length(arr), 2)
    left = merge_sort(Enum.take(arr, mid))
    right = merge_sort(Enum.drop(arr, mid))
    merge(left, right)
  end
  def merge(left, right) do
    merge_helper(left, right, [])
  end
  defp merge_helper([], right, acc), do: acc ++ right
  defp merge_helper(left, [], acc), do: acc ++ left

  defp merge_helper([h_left | t_left], [h_right | t_right], acc) do
    if h_left < h_right do
      merge_helper(t_left, [h_right | t_right], acc ++ [h_left])
    else
      merge_helper([h_left | t_left], t_right, acc ++ [h_right])
    end
  end
end

arr = [3, 6, 8, 10, 1, 2, 1]
sorted_arr = MergeSort.merge_sort(arr)
IO.inspect(sorted_arr) # Изход: [1, 1, 2, 3, 6, 8, 10]
```

```
defmodule MergeSort do
  def merge_sort([], do: [])
  def merge_sort([single]), do: [single]
  def merge_sort(arr) do
    mid = div(length(arr), 2)
    left = merge_sort(Enum.take(arr, mid))
    right = merge_sort(Enum.drop(arr, mid))
    merge(left, right)
  end
  def merge(left, right) do
    merge_helper(left, right, [])
  end
  defp merge_helper([], right, acc), do: acc ++ right
  defp merge_helper(left, [], acc), do: acc ++ left

  defp merge_helper([h_left | t_left], [h_right | t_right], acc) do
    if h_left < h_right do
      merge_helper(t_left, [h_right | t_right], acc ++ [h_left])
    else
      merge_helper([h_left | t_left], t_right, acc ++ [h_right])
    end
  end
end

arr = [3, 6, 8, 10, 1, 2, 1]
sorted_arr = MergeSort.merge_sort(arr)
IO.inspect(sorted_arr) # Изход: [1, 1, 2, 3, 6, 8, 10]
```

[1, 1, 2, 3, 6, 8, 10]

- 1) `defmodule MergeSort do` – В този ред е дефиниран нов модул с име `MergeSort`.
- 2) `def merge_sort([]), do: []` - Тук се определя функцията `merge_sort`, която проверява дали входният списък е празен (`[]`). Ако е така, функцията просто връща празен списък. Това е базов случай за рекурсия.
- 3) `def merge_sort([single]), do: [single]` - Тази версия на `merge_sort` обработва вход, който е списък с един елемент. Ако списъкът съдържа само един елемент `single`, той е вече сортиран, така че функцията просто го връща в списъчна форма, без промени.
- 4) `def merge_sort(arr) do` – От този ред започва основната част на функцията `merge_sort`, която ще обработва списъци с повече от един елемент.
- 5) `mid = div(length(arr), 2)` - Тук се изчислява средния индекс `mid` на списъка `arr`, като използваме `length(arr)` за да получим дължината на списъка и `div/2`, за да получим целочисленото деление. Това ще помогне за разделянето на списъка на две половини.
- 6) `left = merge_sort(Enum.take(arr, mid))`
- Тук се извиква `merge_sort` рекурсивно за лявата половина на списъка. Функцията `Enum.take(arr, mid)` взима елементите от началото на списъка до `middle` индекс (без него), което създава лявата част на списъка.
- 7) `right = merge_sort(Enum.drop(arr, mid))` – Този ред извиква `merge_sort` рекурсивно за дясната половина на списъка. Функцията `Enum.drop(arr, mid)` премахва първите `mid` елемента от списъка `arr`, оставяйки само дясната половина.
- 8) `merge(left, right)` - След рекурсивното сортиране на двете половини, извикваме функцията `merge`, за да ги комбинираме в един сортиран списък. Тази функция ще слее лявата и дясната половина.
- 9) `end` - Тази линия затваря блока на функцията `merge_sort`.

- 10) `def merge(left, right) do` - Тук се декларира функцията `merge`, която приема два списъка - `left` и `right`, които се нуждаят от сливане.
- 11) `merge_helper(left, right, [])` - Вътре в `merge` се извиква помощна функция `merge_helper`, която взема два входни списъка (`left` и `right`) и трети аргумент - акумулиращ списък (`acc`), който се инициализира като празен списък (`[]`). Тази функция извършва действителното сливане на списъците.
- 12) `defp merge_helper([], right, acc), do: acc ++ right` – Този ред определя първия вариант на `merge_helper`, който работи, когато левият списък е празен. В такъв случай, просто се добавят останалите елементи от дясната част (`right`) към акумулиращия списък (`acc`) и се връща.
- 13) `defp merge_helper(left, [], acc), do: acc ++ left` - Тук се определя втория вариант на `merge_helper`, който работи, когато десният списък е празен. В такъв случай, се добавят останалите елементи от левия списък (`left`) към акумулиращия списък (`acc`) и се връща.
- 14) `defp merge_helper([h_left | t_left], [h_right | t_right], acc) do` – Този ред дефинира третия вариант на `merge_helper`, който ще работи, когато и двата списъка имат елементи. Тук е отделя първия елемент на левия списък (`h_left`) и остатъка от него (`t_left`), както и първия елемент на десния списък (`h_right`) и остатъка от него (`t_right`).
- 15) `if h_left < h_right do` - Тук се прави проверка дали текущият най-малък елемент от левия списък (`h_left`) е по-малък от текущия най-малък елемент от десния списък (`h_right`).
- 16) `merge_helper(t_left, [h_right | t_right], acc ++ [h_left])` - Ако условието за минимума на левия списък е вярно, рекурсивно се извиква `merge_helper` с остатъка на левия списък (`t_left`), десния списък (запазихме текущия десен елемент, `h_right`, в началото на списъка) и добавяме `h_left` в акумулиращия списък (`acc`).
- 17) `else merge_helper([h_left | t_left], t_right, acc ++ [h_right])` - Ако условието не е вярно (т.е. `h_right` е по-малък или равен на `h_left`), тогава рекурсивно извикваме `merge_helper` с остатъка от десния списък, запазваме текущия ляв елемент в началото на списъка, и добавяме `h_right` в акумулиращия списък (`acc`).

18) end – Този ред затваря блока на merge_helper.

19) end - Този ред затваря блока на модула MergeSort.

Пример за използване

20) arr = [3, 6, 8, 10, 1, 2, 1] - Тук се декларира списък arr, който съдържа несортирани числа.

21) sorted_arr = MergeSort.merge_sort(arr) - В този ред се извиква функцията merge_sort от модула MergeSort, предавайки списъка arr за сортиране, и съхраняване на резултата (сортирания списък) в променливата sorted_arr.

IO.inspect(sorted_arr) # *Изход: [1, 1, 2, 3, 6, 8, 10]* - И накрая, с тази линия функцията IO.inspect/1 извежда (или "инспектира") стойността на sorted_arr в конзолата, позволявайки ни да видим крайния резултат от сортирането. Очакваният изход е [1, 1, 2, 3, 6, 8, 10].

Обобщение

Този код реализира алгоритъма за сливане (Merge Sort). Алгоритъмът разделя входния списък на две половини, рекурсивно ги сортира и след това ги обединява в един сортиран списък. Използването на помощната функция merge_helper позволява комбинирането на двата сортирани списъка, а функцията merge агрегира резултатите.

3. Пирамидално сортиране (Heapsort)

Изгражда двоично дърво на хип и извежда елементите в сортиран ред. Пирамидалното сортиране изгражда двоично дърво на хип (max-heap или min-heap) от данните и след това систематично извлича елементите от хипа, за да получи сортирания масив.

Времева сложност:

Във всички случаи: $O(n \log n)$

Пример:

```
defmodule HeapSort do
```

```
  def sort(list) do
```

```
    # Строим максимален куп
```

```
    heapified_list = heapify(list)
```

```
    # Сортираме купата
```

```
    sort_heap(heapified_list)
```

```
  end
```

```
  # Изграждане на максимален куп
```

```
  defp heapify(list) do
```

```
    length = length(list)
```

```
    for i <- div(length, 2)..0, reduce: list do
```

```
      acc -> sift_down(acc, i, length)
```

```
    end
```

```
  end
```

```
  # Възстановяване на свойството на купа
```

```
  defp sift_down(list, index, max_index) do
```

```
    left = 2 * index + 1
```

```
    right = 2 * index + 2
```

```
    largest = if left < max_index and Enum.at(list, left) > Enum.at(list, index), do: left, else: index
```

```
    largest = if right < max_index and Enum.at(list, right) > Enum.at(list, largest), do: right, else: largest
```

```
    if largest != index do
```

```
      list = swap(list, index, largest)
```

```
      sift_down(list, largest, max_index)
```

```
    else
```

```
      list
```

```
    end
```

```
  end
```

```
  # Сортиране на купа
```

```
  defp sort_heap(list) do
```

```
    length = length(list)
```

```
    for i <- (length - 1)..1, reduce: list do
```

```
      acc ->
```

```
        acc = swap(acc, 0, i)
```

```
      елемент
```

```
        sift_down(acc, 0, i)
```

```
      купа
```

```
    end
```

```
  end
```

```
  # Размяна на елементи
```

```
  defp swap(list, i, j) do
```

```
    list
```

```
    |> List.replace_at(i, Enum.at(list, j))
```

```
    |> List.replace_at(j, Enum.at(list, i))
```

```
  end
```

```
end
```

```
list = [41, 101, 41, 5, 5]
```

```
sorted_list = HeapSort.sort(list)
```

```
IO.inspect(sorted_list) # => [5, 5, 41, 41, 101]
```

1) defmodule HeapSort do – Този ред определя нов модул с име HeapSort.

Основна функция за сортиране

2) def sort(list) do - Тук започваме декларирането на основната функция sort, която приема списък list за сортиране.

3) *# Строим максимален Heap*

heapified_list = heapify(list) - Когато функцията sort е извикана, тя първо строи максимален Heap от предоставения списък, като извиква функцията heapify(list) и съхранява резултата в heapified_list.

4) *# Сортираме Heap-a*

sort_heap(heapified_list) - След построяването на Heap, функцията sort_heap се извиква с heapified_list за извършване на сортиране.

5) end - Тази линия затваря функцията sort.

Строене на максимален Heap

6) defp heapify(list) do - Тук се дефинира помощната функция heapify, която е отговорна за изграждането на максимален Heap от дадения списък.

7) length = length(list) – Този ред определя дължината на входния списък и я съхранява в променливата length.

8) **for i <- div(length, 2)..0, reduce: list do** - Тук използваме цикъл for, за да итерираме от индекса div(length, 2) (половината от дължината) до 0, обратен ред. reduce: list задава началната стойност на акумулацията на цикъла.

9) acc -> sift_down(acc, i, length) - За всяка итерация на цикъла, извикваме функцията sift_down, за да възстановим свойството на Heap за текущия индекс i. acc представлява акумулирания списък.

10) end end - Тези редове затварят цикъла и функциите heapify.

Възстановяване на свойството на Heap-a

11) defp sift_down(list, index, max_index) do - Тук се дефинира функцията sift_down, която е отговорна за възстановяване на свойството на максимален Heap. Тя приема списък list, индекс index и max_index (максималния индекс за текущия Heap-a).

12) $\text{left} = 2 * \text{index} + 1$ $\text{right} = 2 * \text{index} + 2$ - Тук изчисляваме индексите на лявото (left) и дясното (right) дете на текущия индекс index в Heap.

13) $\text{largest} = \text{if } \text{left} < \text{max_index} \text{ and } \text{Enum.at}(\text{list}, \text{left}) > \text{Enum.at}(\text{list}, \text{index}), \text{do: left, else: index}$ - Тук използваме структура if, за да проверим дали лявото дете (left) е в границите на купа ($\text{left} < \text{max_index}$) и дали е по-голямо от текущия елемент ($\text{Enum.at}(\text{list}, \text{left}) > \text{Enum.at}(\text{list}, \text{index})$). Ако и двете условия са вярни, largest става left, в противен случай остава index.

14) $\text{largest} = \text{if } \text{right} < \text{max_index} \text{ and } \text{Enum.at}(\text{list}, \text{right}) > \text{Enum.at}(\text{list}, \text{largest}), \text{do: right, else: largest}$ - След това проверяваме десното дете (right) по подобен начин и актуализираме largest, ако right е по-голямо от largest.

15) $\text{if } \text{largest} \neq \text{index} \text{ do}$ - Тук проверяваме дали largest е различно от index, което означава, че елементът на текущия индекс не е най-голям.

16) $\text{list} = \text{swap}(\text{list}, \text{index}, \text{largest})$ - Ако largest не е текущият индекс, то размяната на елементите се извършва, за да поставим най-големия елемент на текущата позиция. За това се извиква функцията swap.

17) $\text{sift_down}(\text{list}, \text{largest}, \text{max_index})$ - След размяната, sift_down се извиква рекурсивно за новата позиция на най-голямото дете (largest), за да продължим да възстановяваме свойството на Heap-a.

18) else list - Ако largest е равно на index, то текущото поддържане на Heap-a е успешно и списъкът се връща непроменен.

19) end - Този ред затваря функцията sift_down.

Сортиране на Heap-a

20) $\text{defp sort_heap}(\text{list}) \text{ do}$ Тук започва определението на функцията sort_heap, която е отговорна за сортирането на списъка, представящ Heap-a.

21) $\text{length} = \text{length}(\text{list})$ Определяме дължината на списъка list.

22) $\text{for } i <- (\text{length} - 1)..1, \text{reduce: list do}$ - Тук използваме цикъл for, за да итерираме от последния индекс на Heap-a ($\text{length} - 1$) до 1.

23) $\text{acc} \rightarrow \text{acc}$ представлява текущия акумулиращ списък.

24) $\text{acc} = \text{swap}(\text{acc}, 0, i)$ # *Разменяме корена с последния елемент*

Вътре в цикъла размяната се извършва между корена на Heap ($\text{acc}[0]$) и текущия последен елемент ($\text{acc}[i]$).

25) $\text{sift_down}(\text{acc}, 0, i)$ # *Възстановяване на свойството на Heap.*

След размяната извикваме sift_down за корена, за да възстановим свойството на Heap-a за промения под Heap-a, който включва само елементи до текущия индекс i.

26) end end - Тези редове затварят цикъла и функцията sort_heap.

Размяна на елементи

27) `defp swar(list, i, j) do` – Този ред определя функцията `swar`, която извършва размяна на елементи в списъка.

28) `list |> List.replace_at(i, Enum.at(list, j)) |> List.replace_at(j, Enum.at(list, i))` - Тук `swar` взема списъка и заменя елементите на индекси `i` и `j`, така че стойността на `i` да бъде заменена със стойността на `j`, и обратно, с помощта на `List.replace_at/3`.

29) `end end` – Тези редове затварят блока на функцията `swar` и модула `HeapSort`.

Пример за използване

30) `list = [41, 101, 41, 5, 5]` - Тук декларираме списък `list`, който съдържа несортирани числа.

31) `sorted_list = HeapSort.sort(list)`. В този ред извикваме функцията `sort` от модула `HeapSort`, предавайки списъка `list` за сортиране, и съхраняваме резултата (сортирания списък) в променливата `sorted_list`.

32) `IO.inspect(sorted_list) # => [5, 5, 41, 41, 101]`. И накрая, с този ред функцията `IO.inspect/1` се извежда стойността на `sorted_list` в конзолата, позволявайки ни да видим крайния резултат от сортирането.

Резултат: [5, 5, 41, 41, 101].

Обобщение

Този код реализира алгоритъм за сортиране с помощта на `Heap Sort`. Алгоритъмът строи максимален куп от входния списък, след което последователно премества корена на купа накрая на списъка и възстановява свойството на купа, докато списъкът не бъде напълно сортиран. Основните компоненти на алгоритъма са строежът на купа (`heapify`), възстановяването на свойството на купа (`sift_down`) и самото сортиране (`sort_heap`).

В Elixir for не е традиционен цикъл, а по-скоро генератор на списък. Вместо да се използва за итерация по структури, for в Elixir е конструкции, която позволява на програмистите да генерират нови списъци, извършвайки трансформации и филтрирания на данни.

Основна структура на for

Синтаксисът на for е следният:

for element <- collection, do: expression - Тук element е текущият елемент, който извличаме от collection, а expression е кодът, който се изпълнява за всеки елемент.

Пример

Генериране на списък с квадратите на числата от 1 до 5 чрез for:

```
squares = for x <- 1..5, do: x * x IO.inspect(squares) # => [1, 4, 9, 16, 25]
```

Филтриране допълнително условие:

```
even_squares = for x <- 1..10, rem(x, 2) == 0, do: x * x IO.inspect(even_squares)  
# => [4, 16, 36, 64, 100]
```

Тук генерираме квадратите само на четните числа от 1 до 10.

1) Генериране на квадратите на числата от 1 до 10

Създайте списък с квадратите на числата от 1 до 10.

```
squares = for x <- 1..10, do: x * x2.
```

2) Филтриране на четни числа

Даден списък с числа, генерирайте списък, който съдържа само четните числа.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
even_numbers = for x <- numbers, rem(x, 2) == 0, do: x3.
```

3) Генериране на списък от символи

Създайте списък от символи за буквите от 'a' до 'j'.

```
letters = for x <- ?a..?j, do: <<x>>4.
```

4) Квадрати на четни числа

Отново използвайте `for`, но този път генерирайте квадратите само на четните числа от 1 до 20.

```
even_squares = for x <- 1..20, rem(x, 2) == 0, do: x * x5.
```

5) Генериране на факториели

Създайте списък с факториелите на числата от 1 до 5.

```
factorials = for x <- 1..5, do: Enum.reduce(1..x, 1, &(&1 * &2))
```

6) Преобразуване на списък от стрингове до числа

Даден списък със стрингове, преобразувайте всеки стринг в число.

```
string_numbers = ["1", "2", "3", "4", "5"]  
numbers = for x <- string_numbers, do: String.to_integer(x)
```

7) Преобразуване на низове в Големи букви

Даден списък с низове, генерирайте нов списък с преведени в главни букви низове.

```
words = ["hello", "world", "elixir"]  
uppercased = for word <- words, do: String.upcase(word)
```

8) Генериране на дати

Генерирайте списък с дати за следващите 7 дни от днешната дата.

```
today = Date.utc_today() dates = for x <- 0..6, do: Date.add(today, x)
```

9.) Сумата на числата

Създайте списък със сумирането на числата от 1 до 10.

```
sums = for x <- 1..10, do: Enum.sum(1..x)
```

10) Извеждане на индекси на четни числа

Даден списък от числа, генерирайте нов списък с индекси на четните числа от оригиналния списък.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] even_indices = for {num, index} <- Enum.with_index(numbers),  
rem(num, 2) == 0, do: index
```

Обобщение

В Elixir, `for` не е цикъл в класическия смисъл, а по-скоро инструмент за генериране на нови списъци, базиран на итериране и филтриране на съществуващи колекции. Тъй като Elixir е функционален език, итерациите и трансформациите често са реализирани с функции от модулите `Enum` и `Stream`, което е подходящо за работа с колекции.

Тема 5. Оператори и макроси. Създаване на потребителски функции. Обръщение към функция. Вградени функции.

Използвана литература:

- [1] <https://coderlessons.com/tutorials/kompiuternoe-programmirovanie/vyuchit-eliksir/eliksir-operator>
- [2] <https://elixir-lang.readthedocs.io/en/latest/intro/3.html>
- [3] SAŠA JURIC, “Elixir in Action”, Second Edition, May 2015, ISBN 9781617292019
- [4] <https://gleam.run/cheatsheets/gleam-for-elixir-users/>
- [5] <https://devato.com/post/getting-started-with-elixir-functions>
- [6] <https://elixirschool.com/en/lessons/basics/modules/>
- [7] <https://elixir-examples.github.io/single-page>
- [8] <https://elixir-lang.org/getting-started/meta/quote-and-unquote.html>